Why does this matter?

- What does it mean to say that an operational semantics or implementation is correct?
- How can we reason about the validity of program optimisations?

Release busy?

```
when(busy, idle) { A
  val r = busy.use();
  idle.use(r)
  when(log) { B }
  idle.use(r)
}
when(busy) { C
  when(log) { D }
}
```

Can we release cowns that are highly contended?

Can we reorder behaviours?

How do we address it?

- We could develop a different operational semantics?

  This would making the scheduling of behaviours more complex

  We still wouldn’t have a design of BoC independent of the machinery that realises it

  We may end up being too restrictive or permissive again

- Can we take inspiration from the way others have formulated these designs, namely weak memory models?

  Can we construct an axiomatic model of concurrency that separates intrinsic and incidental causality?